a. Names of GenAI's used: Gemini and ChatGPT
b. Both were accessed on my laptop while a web browser
c. Prompt: "Create the program that is instructed at Assignment 1. Make sure every line is commented, explaining what it does." Also uploaded a PDF of program instructions and requirements
d. Code from Gemini:

```
#include <iostream> // Authored by Gemini: Include standard I/O stream library for printing.

#include <fstream>  // Authored by Gemini: Include file stream library for reading puzzle files.

#include <string>   // Authored by Gemini: Include string library for handling file names.

#include <vector>   // Authored by Gemini: Include vector library to store the 9x9 grid.


using namespace std; // Authored by Gemini: Use standard namespace to avoid typing std:: constantly.


// Authored by Gemini: Define the SudokuSolver class to adhere to object-oriented requirements.

class SudokuSolver {

private:

   int grid[9][9];      // Authored by Gemini: Declare a 2D array to hold the 9x9 Sudoku grid.

   string filename;     // Authored by Gemini: Declare a string to store the name of the puzzle file.

   int solutionCount;    // Authored by Gemini: Declare an integer to track the number of solutions found.


   // Authored by Gemini: Helper function to check if placing 'num' at grid[row][col] is valid.

   bool isValid(int row, int col, int num) {

      for (int i = 0; i < 9; i++) { // Authored by Gemini: Loop 9 times to check the current row and column.
```

```
        if (grid[row][i] == num) return false; // Authored by Gemini: If 'num' exists in the row, return false.
        if (grid[i][col] == num) return false; // Authored by Gemini: If 'num' exists in the column, return false.
      }
      int startRow = row - row % 3; // Authored by Gemini: Calculate the starting row index of the 3x3 block.
      int startCol = col - col % 3; // Authored by Gemini: Calculate the starting column index of the 3x3 block.
      for (int i = 0; i < 3; i++) { // Authored by Gemini: Loop 3 times for the block rows.
        for (int j = 0; j < 3; j++) { // Authored by Gemini: Loop 3 times for the block columns.
          if (grid[i + startRow][j + startCol] == num) return false; // Authored by Gemini: If 'num' exists in the 3x3 block, return false.
        }
      }
      return true; // Authored by Gemini: If no constraints are violated, returning true means the placement is valid.
    }

    // Authored by Gemini: Recursive depth-first search function to solve the puzzle.
    void solveRecursive() {
      int row = -1; // Authored by Gemini: Initialize row variable to -1 indicating no empty cell found yet.
      int col = -1; // Authored by Gemini: Initialize col variable to -1 indicating no empty cell found yet.
      bool emptyLeft = false; // Authored by Gemini: Flag to track if we found an empty cell.

      for (int i = 0; i < 9; i++) { // Authored by Gemini: Iterate through all 9 rows.
```

```
        for (int j = 0; j < 9; j++) { // Authored by Gemini: Iterate through all 9 columns.
            if (grid[i][j] == 0) { // Authored by Gemini: Check if the current cell is empty (represented by 0).
                row = i; // Authored by Gemini: Save the row index of the empty cell.
                col = j; // Authored by Gemini: Save the column index of the empty cell.
                emptyLeft = true; // Authored by Gemini: Set flag to true since an empty cell was found.
                break; // Authored by Gemini: Break out of the inner loop.
            }
        }
        if (emptyLeft) break; // Authored by Gemini: Break out of the outer loop if an empty cell was found.
    }

    if (!emptyLeft) { // Authored by Gemini: If no empty cells are left, the puzzle is solved.
        solutionCount++; // Authored by Gemini: Increment the count of solutions found.
        cout << "Solution " << solutionCount << ":" << endl; // Authored by Gemini: Print the solution number.
        printGrid(); // Authored by Gemini: Call the helper function to print the solved grid.
        return; // Authored by Gemini: Backtrack to find any other potential solutions.
    }

    for (int num = 1; num <= 9; num++) { // Authored by Gemini: Try placing digits 1 through 9 in the empty cell.
        if (isValid(row, col, num)) { // Authored by Gemini: Check if placing 'num' here violates any Sudoku rules.
            grid[row][col] = num; // Authored by Gemini: Place the valid candidate number in the cell.
```

```
            solveRecursive(); // Authored by Gemini: Recursively call the function to attempt filling the next cells.
            grid[row][col] = 0; // Authored by Gemini: Backtrack by resetting the cell to empty (0) if the path fails.
        }
    }
  }

  // Authored by Gemini: Helper function to print the current state of the 9x9 grid.
  void printGrid() {
    for (int i = 0; i < 9; i++) { // Authored by Gemini: Loop through all 9 rows.
      for (int j = 0; j < 9; j++) { // Authored by Gemini: Loop through all 9 columns.
        if (grid[i][j] == 0) cout << "_ "; // Authored by Gemini: Print an underscore for empty cells.
        else cout << grid[i][j] << " "; // Authored by Gemini: Print the digit for filled cells.
      }
      cout << endl; // Authored by Gemini: Print a newline at the end of each row.
    }
    cout << endl; // Authored by Gemini: Print an extra newline for spacing.
  }

public:
  // Authored by Gemini: Constructor to initialize variables.
  SudokuSolver(string file) {
    filename = file; // Authored by Gemini: Assign the passed file name to the class member variable.
    solutionCount = 0; // Authored by Gemini: Initialize the solution counter to zero.
```

```
    for (int i = 0; i < 9; i++) { // Authored by Gemini: Loop to initialize rows.
        for (int j = 0; j < 9; j++) { // Authored by Gemini: Loop to initialize columns.
            grid[i][j] = 0; // Authored by Gemini: Set all initial grid values to 0.
        }
    }
}

// Authored by Gemini: Method to read the puzzle from the provided text file.
bool loadPuzzle() {
    ifstream file(filename); // Authored by Gemini: Open the file stream for the given filename.
    if (!file.is_open()) { // Authored by Gemini: Check if the file failed to open.
        cout << "Error opening file: " << filename << endl; // Authored by Gemini: Print an error message if the file doesn't exist.
        return false; // Authored by Gemini: Return false indicating failure to load.
    }
    char ch; // Authored by Gemini: Declare a character variable to read from the file cell by cell.
    for (int i = 0; i < 9; i++) { // Authored by Gemini: Loop through the 9 rows of the puzzle file.
        for (int j = 0; j < 9; j++) { // Authored by Gemini: Loop through the 9 columns of the puzzle file.
            if (file >> ch) { // Authored by Gemini: Attempt to read the next non-whitespace character.
                if (ch == '_') { // Authored by Gemini: Check if the character is an underscore representing a blank.
                    grid[i][j] = 0; // Authored by Gemini: Store 0 in the grid array to represent the blank.
```

```
            } else if (ch >= '1' && ch <= '9') { // Authored by Gemini: Check if the character is a digit from 1 to 9.
                grid[i][j] = ch - '0'; // Authored by Gemini: Convert the char digit to an integer and store it.
            }
        }
    }
    file.close(); // Authored by Gemini: Close the file stream to prevent memory leaks.
    return true; // Authored by Gemini: Return true indicating the puzzle was successfully loaded.
}

// Authored by Gemini: Public method to trigger the solver and format the required output.
void solve() {
    cout << "File: " << filename << endl; // Authored by Gemini: Print the puzzle file name as required by the assignment.
    cout << "Original Puzzle:" << endl; // Authored by Gemini: Label the printing of the original puzzle state.
    printGrid(); // Authored by Gemini: Print the initial unmodified puzzle to the console.

    solveRecursive(); // Authored by Gemini: Call the recursive search method to find all solutions.

    if (solutionCount == 0) { // Authored by Gemini: Check if the recursive search found zero solutions.
        cout << "No solution found" << endl; // Authored by Gemini: Print the required failure string if no solution exists.
```

```
    }
    cout << "-----------------------------------" << endl; // Authored by Gemini: Print a separator for readability between files.
  }
};

// Authored by Gemini: Main execution block.
int main(int argc, char* argv[]) { // Authored by Gemini: Entry point accepting command-line arguments.
  if (argc > 1) { // Authored by Gemini: If command-line arguments are provided (argc > 1).
    for (int i = 1; i < argc; i++) { // Authored by Gemini: Loop through all provided file name arguments.
      SudokuSolver solver(argv[i]); // Authored by Gemini: Instantiate a SudokuSolver object for the current file.
      if (solver.loadPuzzle()) { // Authored by Gemini: Attempt to load the puzzle data into the object.
        solver.solve(); // Authored by Gemini: Solve the loaded puzzle and print the output.
      }
    }
  } else { // Authored by Gemini: If no command-line arguments are provided, use traditional input.
    string filename; // Authored by Gemini: Declare a string variable to capture user input.
    cout << "Enter the puzzle filename (e.g., puzzle1.txt): "; // Authored by Gemini: Prompt the user for a single filename.
    cin >> filename; // Authored by Gemini: Read the filename from standard input.
    SudokuSolver solver(filename); // Authored by Gemini: Instantiate a SudokuSolver object for the entered file.
    if (solver.loadPuzzle()) { // Authored by Gemini: Attempt to load the puzzle data into the object.
```

```
        solver.solve(); // Authored by Gemini: Solve the loaded puzzle and print the output.
      }
    }
    return 0; // Authored by Gemini: Terminate the program cleanly.
}
```

Code form ChatGPT:

```
// Include file-stream tools so the program can read puzzle files.
#include <fstream> // Provides std::ifstream for opening and reading files.
#include <iostream> // Provides std::cout and std::cerr for displaying output and errors.
#include <string> // Provides std::string for filenames and input text.
#include <vector> // Provides std::vector for storing the grid and solutions.
#include <cctype> // Provides std::isspace for ignoring whitespace in puzzle files.

// Define a class that keeps the Sudoku grid and solving behavior together.
class SudokuSolver { // Start of the object-oriented Sudoku solver class.
private: // Keep the grid and helper methods internal to this class.
    static constexpr int SIZE = 9; // Store the number of rows and columns in a Sudoku grid.
    static constexpr int BOX_SIZE = 3; // Store the width and height of each 3-by-3 box.
    std::vector<std::vector<int>> grid; // Store each cell as 1-9, with 0 representing a blank.
    std::vector<std::vector<std::vector<int>>> solutions; // Store every completed solution found.

    bool isSafe(int row, int col, int value) const { // Check whether a value can legally go in one cell.
        for (int index = 0; index < SIZE; ++index) { // Visit each position in the row and column.
```

```
            if (grid[row][index] == value) { // Check whether this value already appears in the row.
                return false; // Reject the value because Sudoku rows cannot repeat digits.
            } // End the row-conflict check.
            if (grid[index][col] == value) { // Check whether this value already appears in the column.
                return false; // Reject the value because Sudoku columns cannot repeat digits.
            } // End the column-conflict check.
        } // Finish checking the complete row and column.
        int boxStartRow = (row / BOX_SIZE) * BOX_SIZE; // Find the first row of this cell's 3-by-3 box.
        int boxStartCol = (col / BOX_SIZE) * BOX_SIZE; // Find the first column of this cell's 3-by-3 box.
        for (int r = boxStartRow; r < boxStartRow + BOX_SIZE; ++r) { // Visit all rows in this box.
            for (int c = boxStartCol; c < boxStartCol + BOX_SIZE; ++c) { // Visit all columns in this box.
                if (grid[r][c] == value) { // Check whether the box already contains this value.
                    return false; // Reject the value because a box cannot repeat a digit.
                } // End the box-conflict check.
            } // Finish this row of the box.
        } // Finish checking every cell in the box.
        return true; // Report that the value does not conflict with any Sudoku rule.
    } // End of isSafe.

    bool initialGridIsValid() { // Verify that the puzzle's pre-filled digits obey Sudoku rules.
        for (int row = 0; row < SIZE; ++row) { // Visit every row in the starting puzzle.
            for (int col = 0; col < SIZE; ++col) { // Visit every cell in the current row.
```

```
            int value = grid[row][col]; // Read the value currently stored in this cell.
            if (value == 0) { // Determine whether this cell is an empty cell.
                continue; // Skip empty cells because they do not conflict with a digit.
            } // Finish handling an empty cell.
            grid[row][col] = 0; // Temporarily clear this cell so it will not conflict with itself.
            bool valid = isSafe(row, col, value); // Check the digit against the rest of its row, column, and box.
            grid[row][col] = value; // Restore the original pre-filled digit after checking it.
            if (!valid) { // Check whether the starting puzzle contains a duplicate digit.
                return false; // Mark the puzzle invalid because its given clues conflict.
            } // Finish handling a conflicting starting digit.
        } // Finish checking the current row.
    } // Finish checking every starting cell.
    return true; // Confirm that all pre-filled digits are consistent.
} // End of initialGridIsValid.

void searchForSolutions() { // Recursively search for all solutions using backtracking.
    int emptyRow = -1; // Use -1 to indicate that no empty cell has been found yet.
    int emptyCol = -1; // Use -1 to indicate that no empty cell has been found yet.
    bool foundEmpty = false; // Track whether the grid still has an unfilled cell.
    for (int row = 0; row < SIZE && !foundEmpty; ++row) { // Search rows until an empty cell is located.
        for (int col = 0; col < SIZE; ++col) { // Search each column in the current row.
            if (grid[row][col] == 0) { // Identify an unfilled cell.
                emptyRow = row; // Remember the row of the empty cell.
                emptyCol = col; // Remember the column of the empty cell.
                foundEmpty = true; // Record that a blank cell has been found.
                break; // Stop scanning this row because a cell is ready to fill.
```

```
            } // Finish checking this cell.
        } // Finish scanning the current row.
    } // Finish looking for an empty cell.
    if (!foundEmpty) { // Determine whether every cell has been filled legally.
        solutions.push_back(grid); // Save a copy of this complete solution.
        return; // End this recursive path because the grid is solved.
    } // Finish handling a completed grid.
    for (int value = 1; value <= SIZE; ++value) { // Try each possible Sudoku digit from 1 through 9.
        if (isSafe(emptyRow, emptyCol, value)) { // Only place a digit if it obeys all Sudoku constraints.
            grid[emptyRow][emptyCol] = value; // Tentatively place this candidate in the empty cell.
            searchForSolutions(); // Recursively try to complete the grid with this candidate.
            grid[emptyRow][emptyCol] = 0; // Undo the tentative choice to try another candidate.
        } // Finish testing this candidate digit.
    } // Finish trying all possible digits for this empty cell.
} // End of searchForSolutions.


void printGrid(const std::vector<std::vector<int>>& board) const { // Print a grid in a readable Sudoku layout.
    for (int row = 0; row < SIZE; ++row) { // Print the grid one row at a time.
        if (row > 0 && row % BOX_SIZE == 0) { // Check whether a horizontal box divider belongs before this row.
            std::cout << "------+-------+------\n"; // Separate the three horizontal bands of boxes.
        } // Finish checking for a horizontal divider.
        for (int col = 0; col < SIZE; ++col) { // Print every cell in the current row.
```

```
            if (col > 0 && col % BOX_SIZE == 0) { // Check whether a vertical box divider belongs here.
                std::cout << "| "; // Separate the three vertical boxes.
            } // Finish checking for a vertical divider.
            if (board[row][col] == 0) { // Check whether the cell is blank in the original puzzle.
                std::cout << "_ "; // Show an underscore for an empty cell.
            } else { // Handle a cell containing a digit.
                std::cout << board[row][col] << ' '; // Print the digit followed by a space.
            } // Finish printing this cell.
        } // Finish printing all cells in this row.
        std::cout << '\n'; // Move to the next output line after this row.
    } // Finish printing all nine rows.
} // End of printGrid.


public: // Expose the operations that main needs to use.
    SudokuSolver() : grid(SIZE, std::vector<int>(SIZE, 0)) {} // Start with a 9-by-9 grid filled with zeros.


    bool loadFromFile(const std::string& filename) { // Load and validate the basic format of a puzzle file.
        std::ifstream inputFile(filename); // Attempt to open the requested puzzle file.
        if (!inputFile) { // Check whether the file could not be opened.
            std::cerr << "Error: could not open " << filename << ".\n"; // Explain which file failed to open.
            return false; // Tell the caller that loading failed.
        } // Finish handling an open failure.
        std::vector<int> values; // Collect the 81 puzzle cells in reading order.
        char character; // Hold each character read from the input file.
```

```
        while (inputFile.get(character)) { // Read the file one character at a time.
            if (std::isspace(static_cast<unsigned char>(character))) { // Ignore spaces, tabs, and newline characters.
                continue; // Move to the next character without treating whitespace as a cell.
            } // Finish skipping whitespace.
            if (character == '_') { // Check whether the character marks a blank cell.
                values.push_back(0); // Store zero internally to represent the blank.
            } else if (character >= '1' && character <= '9') { // Check whether the character is a valid Sudoku digit.
                values.push_back(character - '0'); // Convert the character digit into its integer value.
            } else { // Handle any character that is not whitespace, underscore, or digit 1-9.
                std::cerr << "Error: invalid character in " << filename << ".\n"; // Identify the file containing bad input.
                return false; // Stop loading because the puzzle format is invalid.
            } // Finish interpreting this input character.
            if (values.size() > SIZE * SIZE) { // Check whether the file contains more than 81 cell symbols.
                std::cerr << "Error: " << filename << " contains more than 81 cells.\n"; // Explain the excessive cell count.
                return false; // Reject a puzzle with too many cells.
            } // Finish checking the maximum number of cells.
        } // Finish reading the complete file.
        if (values.size() != SIZE * SIZE) { // Check whether exactly 81 cells were read.
            std::cerr << "Error: " << filename << " must contain exactly 81 cells.\n"; // Explain the required puzzle size.
            return false; // Reject a puzzle that has too few or too many cells.
        } // Finish checking the cell count.
        for (int row = 0; row < SIZE; ++row) { // Copy values into each row of the internal grid.
```

```
            for (int col = 0; col < SIZE; ++col) { // Copy values into each column of this row.
                grid[row][col] = values[row * SIZE + col]; // Store the matching input cell in the 9-by-9 grid.
            } // Finish filling the current row.
        } // Finish filling the entire grid.
        return true; // Report that the puzzle file was loaded successfully.
    } // End of loadFromFile.

    void solveAndPrint(const std::string& filename) { // Solve the loaded puzzle and print its results.
        solutions.clear(); // Remove solutions from any previous puzzle solved by this object.
        std::cout << "\nPuzzle file: " << filename << "\n"; // Display the name of the current puzzle file.
        std::cout << "Original puzzle:\n"; // Label the puzzle as it appeared in the file.
        printGrid(grid); // Display the starting puzzle before attempting to solve it.
        if (!initialGridIsValid()) { // Reject a starting grid with conflicting pre-filled digits.
            std::cout << "No solution found (the given clues conflict).\n"; // Report that the initial puzzle is invalid.
            return; // Skip the search because conflicting clues cannot produce a valid solution.
        } // Finish validating the starting puzzle.
        searchForSolutions(); // Run recursive depth-first search with backtracking.
        if (solutions.empty()) { // Check whether the search failed to find any complete solution.
            std::cout << "No solution found.\n"; // Print the required message when no solution exists.
            return; // Finish processing this puzzle.
        } // Finish handling the no-solution case.
```

```
        std::cout << "Number of solutions found: " << solutions.size() << "\n"; // Report how many solutions were discovered.

        for (std::size_t index = 0; index < solutions.size(); ++index) { // Visit every solution found by the search.

            std::cout << "\nSolution " << index + 1 << ":\n"; // Label the next solution using a human-friendly number.

            printGrid(solutions[index]); // Print the current completed Sudoku grid.

        } // Finish printing all solutions for this puzzle.

    } // End of solveAndPrint.

}; // End of the SudokuSolver class.


// Define the program's entry point, where execution begins.

int main() { // Start the main function.

    const std::vector<std::string> filenames = { // Create the list of puzzle files required by the assignment.

        "puzzle1.txt", // Name of the first Sudoku puzzle file.

        "puzzle2.txt", // Name of the second Sudoku puzzle file.

        "puzzle3.txt", // Name of the third Sudoku puzzle file.

        "puzzle4.txt", // Name of the fourth Sudoku puzzle file.

        "puzzle5.txt" // Name of the fifth Sudoku puzzle file.

    }; // Finish defining the required filenames.

    SudokuSolver solver; // Create one solver object to process each puzzle in sequence.

    for (const std::string& filename : filenames) { // Process each required puzzle file.

        if (solver.loadFromFile(filename)) { // Continue only if the current file was read successfully.

            solver.solveAndPrint(filename); // Print the original puzzle and every solution found.

        } else { // Handle a file that could not be loaded or had an invalid format.
```

```
            std::cout << "Skipping " << filename << " because it could not be loaded.\n"; // Explain why this puzzle was skipped.

        } // Finish handling the current file's loading result.

    } // Finish processing all five required puzzle files.

    return 0; // Tell the operating system that the program ended successfully.

} // End of main.
```

e. (Used Claude to test code)
   a. Correctness: Both programs compiled and ran with no issues on all five puzzle files. They produced identical results: one solution each for puzzles 1, 2 and 3, no solution for puzzle 4 (cells (5,3) and (6,1) are both forced to be 6 in the same box), and two solutions for puzzle 5. The difference is in how they handle bad input. ChatGPT reports an error and skips the file when a puzzle contains an invalid character or doesn't have exactly 81 cells, and it detects conflicting given clues and says so. Gemini fails silently in these cases. An invalid character x was treated as a blank, so it solved a different puzzle than the file described. A file truncated to five rows was accepted and solved with four blank rows. Gemini also never validates the given clues, though it still reported "No solution found" in my duplicate-5 test. Neither program crashed, but only ChatGPT gives meaningful error messages, so ChatGPT is more correct on messy input.

   b. Execution time: Both programs use the same algorithm (find the first empty cell, try 1–9, recurse, backtrack), so both are worst-case O(9^m) for m empty cells, and each recursive call also rescans the grid for the next empty cell. I timed each solver in-process over 2,000 runs. Without optimization (-O0), Gemini was about 3× faster (for example, 0.245 ms vs 0.929 ms on puzzle 5), because its flat int grid[9][9] avoids the extra pointer indirection of ChatGPT's vector<vector<int>>. With -O2, the gap mostly disappeared. Gemini was about 20–35% faster on the easy puzzles and about equal on puzzle 5 (0.090 ms vs 0.092 ms). All five puzzles solve in under 1 ms, so the differences are constant-factor only, and Gemini is slightly faster.

c. Space complexity: Both programs use a fixed 9×9 grid and a recursion stack of at most 81 frames, so both are O(1) with respect to the grid. They differ in how they handle solutions. ChatGPT stores every solution in a vector before printing, which is O(S × 81) for S solutions, and each stored grid costs about ten heap allocations. Gemini prints each solution as it is found, so its extra space is O(1). ChatGPT's vector<vector<int>> grid and its temporary 81-element loading vector also add overhead. This comparison comes from static analysis of the code, and I did not measure memory. For your five puzzles (at most two solutions) the difference is negligible, but Gemini's design has no memory growth and is the better one.

d. Maintainability: ChatGPT is easier to maintain. It uses named constants (SIZE, BOX_SIZE), const methods, and explicit std:: instead of using namespace std. It separates loading, clue validation, search and printing into distinct methods, and its search returns solutions instead of printing them, which makes it easier to test and extend. It also has a complete prolog and accurate comments, though it is over-commented (even closing braces get comments) and hardcodes the five filenames in main. Gemini is more compact and accepts filenames on the command line, but it uses magic numbers (9 and 3) throughout, using namespace std, a recursive search that prints directly, and a main that duplicates its load-and-solve logic in two branches. Its "Authored by Gemini" tag on every line is repetitive, and a few comments are inaccurate (for example, closing the file "to prevent memory leaks"). Its weak error handling also makes failures harder to diagnose.

f. I would choose ChatGPT. Both programs gave identical, correct answers on all five of the puzzles, so the deciding factor is how they behave when something goes wrong. ChatGPT rejects invalid characters and files that don't have exactly 81 cells, and it detects conflicting clues, while Gemini silently treated a bad x as a blank and solved a truncated puzzle as if it were valid. ChatGPT is also easier to maintain, thanks to named constants, const methods, and a search that returns solutions instead of printing them. It does store every solution, and it was slower in unoptimized builds, but in puzzles this small both costs are negligible compared with the reliability and structure it provides.

g. I improved the program in four areas. For correctness, the loader now reports the exact row and column of a bad character, a wrong cell count, or a conflicting clue, and a bad file can no longer leave a half-loaded puzzle behind. For speed, the search tracks used digits with bitmasks and always fills the most-constrained cell first, which made a hard puzzle about 34× faster while producing the same solutions on all my tests. For space, solutions are printed as they are found instead of being stored, so memory stays flat even on an all-blank grid. For maintainability, the search now reports solutions through a callback instead of printing, and filenames can be passed on the command line.